Problem A. Records in Chichén Itzá

Consider the number of non-leaf vertices, that is, vertices whose degree is at least 2. There are only a few cases.

- If there are at most two non-leaf vertices, the tree is a star or a double star obtained by joining the centers of two stars. In either case the tree is unique.
- If there are at least three non-leaf vertices and their degrees are not all equal, then there are at least two different orders in which three of those vertices can be joined. Hence the tree is not unique.
- If there are at least four non-leaf vertices and their degrees are all equal but not equal to 2, then four such vertices can already be connected in at least two different structures, so the tree is not unique.
- In all remaining cases, the non-leaf vertices form a unique chain. Therefore the entire tree is unique.

This gives a direct characterization after counting degrees.

Problem B. Almost Convex 2

Choose the lexicographically smallest point and call it p_1. Count, for all triangles $p_1p_ip_j$, the number of given points strictly inside them. This can be done in $\mathcal{O}(n^3)$ time, and then it lets us decide in $\mathcal{O}(1)$ time whether a triangle $p_kp_ip_j$ contains any point in its interior.

Now consider the possible value of $|Q|$, where R denotes the convex hull.

- If $|Q| = |R|$, then Q is exactly the convex hull R.
- If $|Q| = |R| + 1$, cut an edge vw of R and pick a point $u \notin R$ such that the triangle uvw contains no point in its interior. Replace the edge vw by uv and uw. There are $\mathcal{O}(n^2)$ candidates.
- If $|Q| = |R| + 2$, start from a polygon of the previous type, cut an edge $v'w'$ of it, and insert a point $u' \notin Q$ such that triangle $u'v'w'$ has no interior point. The resulting polygon must still be simple.

For the last case, there are two subcases.

- If $v'w'$ is not an edge of the initial hull R, there are only two such edges. Enumerating the edge and u' costs $\mathcal{O}(n)$ for each of the $\mathcal{O}(n^2)$ previous candidates, so the total is $\mathcal{O}(n^3)$.
- If $v'w'$ is an edge of the initial hull R, the problem is equivalent to choosing two empty triangles uvw and $u'v'w'$ based on hull edges, with the additional condition that the two triangles do not intersect. When u and u' are fixed, the line through them separates R into two sides; the two triangles can intersect only if vw and $v'w'$ lie on the same side.

Thus we can enumerate vw and maintain the valid range of $v'w'$ with two pointers. The time complexity is $\mathcal{O}(n^3)$.

Problem C. Space Station

The linked source currently says that this analysis will be added later. No additional solution text for this problem is present in the source post.

Problem D. Solar Panel Grid Optimization

First consider the following procedure. Enumerate each column c from left to right, for $1 \le c < n$. For every row i satisfying $a_{n,c} = b_{i,c}$, perform `row i`. Then perform `column n` exactly n times. Finally, perform `row i` for the rows that were not operated on before. After completing this procedure for a column c, column $n - 1$ of a becomes equal to column c of b.

After all these operations, the last column may still fail to match. Observe that the parity of the number of ones in each row changes in a fixed way during the above procedure: a `row` operation does not change the parity of that row, while n consecutive `column n` operations change the parity of every row simultaneously.

Therefore it suffices to first make the row parities of a and b either identical or opposite, depending on the parity of n.

From this, compute the desired final value of each $a_{i,n}$, and denote it by A_i. It remains to adjust only these parities.

We first make both the n-th row and the n-th column all zero, and then reconstruct the target parity pattern from all zeroes. Since complementing all zeroes and ones is essentially symmetric, assume that among the $2n-1$ cells in the n-th row and n-th column, there are more ones than zeroes.

To clear the n-th column, repeat the following n times: perform `row n` until $a_{n,n} = 1$, and then perform `column n`. This turns n ones into zeroes and places them into the last column. Next, clear the n-th row by repeatedly moving a one in that row to position (n, n) using `row n`, and then applying `column n`.

At this point, perform `column n` n times so that the last column becomes all ones. Ignoring A_n for the moment, solve $A_1, \ldots, A_{n-1}$. As in the first procedure, process $A_{n-1}, A_{n-2}, \ldots, A_1$. If the target value is 1, pull a zero from the last row and then perform `column n`; otherwise, directly perform `column n`. The remaining parity of the last row is easy to fix.

The first part uses $2n(n-1)$ operations, and the parity adjustment uses only $\mathcal{O}(n)$ additional operations, which is within the required bound.

Problem E. Fast Median Transform

First consider how to answer a single query $\mathrm{FMT}(a, b, X)$.

A few small subtasks are immediate: one can output 0 in the easiest case, and directly simulate the process in the brute-force case. In general, by binary searching the answer, or by discretizing and then binary searching, the problem can be reduced to the case $V = 2$ at the cost of a factor $\log V$ or $\log(n+m)$.

For $V = 2$, the key observation is simple:

- If an operation has $a_i = b_j = 0$, then it assigns $X \leftarrow 0$.
- If an operation has $a_i = b_j = 1$, then it assigns $X \leftarrow 1$.
- If $a_i \neq b_j$, then the operation does not change X.

Thus only the last assignment operation matters. Some subtasks then become straightforward: in one of them the answer is just X, and in another it is enough to check whether the relevant sequence contains a zero.

Suppose the positions of ones in a are $p_1, p_2, \ldots, p_k$, with $k \leq 5$. Search backward from time $nm-1$ for the last assignment operation. Consider, for example, the last operation involving a_{p_1}, say (a_{p_1}, b_{q_1}). If it is not an assignment, then $b_{q_1} = 0$. Look at the last six operations involving b_{q_1}. Since $\gcd(n, m) = 1$, these six operations touch pairwise different positions of a, and one of them is not among $p_1, \ldots, p_k$. Therefore the last assignment is at distance at most $6(n+m)$ from $nm-1$, so we can determine the answer by going back only that far.

For the next subtask, the same idea can be strengthened. Take the last $d = 2(n+m)$ operations. We prove that either one of them is an assignment, or there is no assignment in the whole process. Choose an index i. In the last d operations there are at least two operations involving a_i, say (a_i, b_{j_1}) and (a_i, b_{j_2}). If $a_i = 1$ and neither operation is an assignment, then $b_{j_1} = b_{j_2} = 0$; the case $a_i = 0$ is symmetric.

Build a bipartite graph from these last d operations: left vertices are indices i of a, right vertices are indices j of b, and an operation creates an edge. Every vertex has degree at least 2. It remains to prove that this bipartite graph is connected. Indeed, if the last d operations contain no assignment, then all left vertices must have the same value and all right vertices the opposite value, so the answer is still the initial X.

Take any connected component, and let S be its set of right vertices. For any $j \in S$, let the last operation involving b_j be (a_i, b_j), and look at the previous operation involving this a_i, say $(a_i, b_{j'})$. This operation also lies among the last d operations, so $j' \in S$, and moreover $j' \equiv j - n \pmod{m}$. Since $\gcd(n, m) = 1$,

repeated jumps prove that S must be the whole set of right vertices. The left side is analogous, so the graph is connected.

Equivalently, the proof uses the elementary fact that for a nonempty set $S \subseteq \{0, 1, \ldots, m-1\}$ and $d \perp m$, the condition $S \equiv S + d \pmod{m}$ holds if and only if S is the whole set.

For the general case, let $c = \gcd(n, m)$. After separately handling the case of no assignment, one can still prove that an assignment operation must appear within the last $2(n + m)$ operations. When n and m are of the same order, this gives a single-query solution in $\mathcal{O}(n \log n)$ or $\mathcal{O}(n \log V)$. A more standard but slower method is an $\mathcal{O}(n \log^2 n)$ approach based on equations after the binary search.

In fact, if we analyze what the binary search is doing, it can be removed: it is enough to simulate the last $2 \max(n, m)$ operations directly. This gives an $\mathcal{O}(n)$ method for a single query.

For the full setting $n, Q \le 5 \cdot 10^5$, recomputing each query is impossible. Let

$$U(x, y) = \begin{cases} [x, y], & x \le y, \\ [y, x], & x > y. \end{cases}$$

If there are no modifications and only X varies, maintain an interval $[L, R]$, initially $[0, +\infty)$. Enumerate the operations backward and intersect the maintained interval with the current interval. If the intersection ever becomes empty, the answer has already been determined. Otherwise the answer is X_0 exactly when $X_0 \in [L, R]$; if not, the answer is the closer boundary L or R according to the side on which X_0 lies. The previous proof shows that only the last $2 \max(n, m)$ operations need to be considered.

The interval transformations form a semigroup. For the next subtask, take the last $2 \max(n, m)$ operations and maintain them with a segment tree. Since $n \ge m$, one update affects at most two operations, giving time $\mathcal{O}((n + Q) \log n)$.

For a square-root decomposition approach, let $N = \max(n, m)$ and choose a threshold B. We use the following interval lemma: for intervals $[l_i, r_i]$, if $L = \max_i l_i$ and $R = \min_i r_i$, then their intersection is empty when $L > R$, and otherwise it is $[L, R]$.

If $n \ge B$, each update changes only $\mathcal{O}(N/B)$ operation records, so a segment tree gives total complexity $\mathcal{O}(N^2 \log N/B)$. If $n \le B$, answer each query in $\mathcal{O}(B \log N)$. For a fixed value a_i, among all paired values b_j, only the smallest $b_j \ge a_i$ and the largest $b_j \le a_i$ can matter. We can find these by $\mathcal{O}(n)$ binary searches and then compute the interval intersection, for total $\mathcal{O}(NB \log N)$.

If the intersection is nonempty, the answer is immediate. Otherwise, find the largest d such that considering only operations with times $i \in [d, nm - 1]$ gives a nonempty intersection; then the answer follows by inspecting operation $d - 1$. Given d, list for every a_i all paired b_j values sorted by time and precompute suffix maxima and minima. During a query, find the first operation at or after d for each a_i and read the needed values in $\mathcal{O}(1)$ each, so the check costs $\mathcal{O}(n)$. Binary searching d costs $\mathcal{O}(n \log N)$. Taking $B = \sqrt{n}$ yields $\mathcal{O}(N\sqrt{N} \log N)$.

Finally, here is the outline of the $\mathcal{O}(N \log N)$ method. Let $MX = nm - 1$ and $L = 2 \max(n, m)$. The case $n \ge m$ is handled as above, so assume $n < m$. Reverse the last L operations. For $i \in [0, L)$, suppose operation $MX - i$ is (a_{p_i}, B_i). Maintain n sequences $A_0, A_1, \ldots, A_{n-1}$ by appending B_i to A_{p_i}. The sequences are deterministic; each is nonempty, their lengths differ by at most one, and the total length is L.

First decide whether all L intervals intersect. For each i,

$$\bigcap_{x \in A_i} U(x, a_i) = U\big(a_i, \max_{x \in A_i,\, x < a_i} x\big) \cap U\big(a_i, \min_{x \in A_i,\, x \ge a_i} x\big).$$

Thus each a_i contributes at most two intervals. Maintaining the maximum left endpoint and minimum right endpoint with multisets costs $\mathcal{O}((n + Q) \log n)$. If the global intersection is nonempty, the answer follows immediately.

Otherwise, find a time T such that

$$\bigcap_{i=0}^{T-1} U(A_{p_i}, B_i) \ne \varnothing,$$

and, if

$$Z = \bigcap_{i=0}^{T} U(A_{p_i}, B_i),$$

then Z is either empty or a single point. Let Y be the intersection before time T; comparing $U(A_{p_T}, B_T)$ with Y determines the answer, which must be an endpoint of Y. A segment tree over the first n intervals handles the easy cases, since one update changes at most one interval.

For the remaining case, let $Z_0 = \bigcap_{i=0}^{n-1} U(A_{p_i}, B_i)$ and suppose it is a non-degenerate interval. For each i, define $c_i = 0$ if $a_i < A_{i,0}$ and $c_i = 1$ otherwise. If there are i, j with $a_i \le a_j$ and $c_i > c_j$, then Z_0 would be empty or a point, contradiction. Hence there is a threshold V such that $c_i = [a_i \le V]$. Maintain

$$LB = \max_{c_i=0} a_i, \qquad RB = \min_{c_i=1} a_i,$$

so always $LB < RB$.

For $t \ge n-1$, write $Z_t = \bigcap_{i=0}^{t} U(A_{p_i}, B_i)$. Then Z_t is non-degenerate if and only if

$$\min\left(RB, \min_{i \le t,\, c_{p_i}=0} B_i\right) > \max\left(LB, \max_{i \le t,\, c_{p_i}=1} B_i\right).$$

Sort indices by $A_{i,0}$ to obtain a permutation q, and let $w = q^{-1}$. Since Z_{n-1} is non-degenerate, there is a K such that $[c_{q_i} = 0] = [i \ge K]$, with $K = \sum_i [c_i = 1]$.

To compute $\min_{i \le t,\, c_{p_i}=0} B_i$, write $t = xn + y$ where $y \in [-1, n-2]$. The part with $i < xn$ becomes

$$\min_{w_i \ge K,\, j < x} A_{i,j},$$

which can be answered in $\mathcal{O}(1)$ after a two-dimensional prefix-minimum preprocessing. The remaining part

$$\min_{xn \le i \le xn+y,\, c_{p_i}=0} B_i = \min_{i \ge n-1-y,\, w_i \ge K} A_{i,x}$$

is a two-dimensional dominance query and can be maintained with persistent segment trees. The maximum for $c_{p_i} = 1$ is symmetric.

This allows testing a candidate t in $\mathcal{O}(\log N)$, so one can find T in $\mathcal{O}(\log^2 N)$, yielding an $\mathcal{O}(N \log^2 N)$ solution. To reduce this to $\mathcal{O}(N \log N)$, first binary search the block x with $xn \le T < (x+1)n$ using $\mathcal{O}(1)$ tests, and then binary search the exact T inside the persistent segment tree.

Problem F. Colorful Graph 3

If some $c_i \ge 2$, a flower-shaped construction using the corresponding color is enough.

Now consider the theoretical minimum number of edges. Let a_i be the number of edges of color i. Define t_i as follows: if $c_i = 0$, then $t_i = 0$; otherwise t_i is the largest integer satisfying

$$t_i(t_i + 1) \le a_i.$$

To guarantee connectivity, for every $1 \le i \le n$ we need

$$\left(\sum_j a_j\right) - a_i + t_i \ge n - 1.$$

Since $a_i - t_i$ is monotone nondecreasing as a_i grows, there exists an optimal solution in which the range of the values $a_i - t_i$ is at most 1.

Assume we have found a minimum feasible vector (a_i). We use two constructions.

- For colors with $c_i = 0$: repeatedly take one edge of each color, form a cycle, and attach it at an arbitrary vertex until no edge remains. After removing any one color, what remains is a collection of connected chains.

- For the exact case $c_i = 1$: let $T = \max_i t_i$. Construct the complete graph on T vertices, and replace every edge by a chain containing one edge of each color.

For the general case, first apply the $c_i = 1$ construction and also insert the $c_i = 0$ edges into the chains of the complete graph, except for a spanning tree. The remaining edge counts differ by at most one, so we can finish with the $c_i = 0$ construction. This construction attains the lower bound.

Problem G. CNOI Knowledge

The linked source currently says that this analysis will be added later. No additional solution text for this problem is present in the source post.

Problem H. Exchanging Kubic 2

The linked source currently says that this analysis will be added later. No additional solution text for this problem is present in the source post.

Problem I. Nightmare

The task is to find n vectors in a vector space over $\mathbb{F}_p$, with dimension m as small as possible, such that the given matrix G is their Gram matrix.

View G as a quadratic form. Suppose G is the Gram matrix of $\{v_1, \ldots, v_n\}$. For

$$x = \sum_{i=1}^{n} x_i v_i,$$

with coordinate vector X, we have

$$X^T G X = \sum_{i=1}^{n} \sum_{j=1}^{n} x_i x_j \langle v_i, v_j \rangle = \langle x, x \rangle.$$

If a congruence transformation changes G to $G' = A^T G A$, where A is the change-of-basis matrix from coordinates in a basis $\{w_1, \ldots, w_n\}$ to coordinates in $\{v_1, \ldots, v_n\}$, then

$$X^T G' X = (AX)^T G (AX) = \langle x, x \rangle.$$

Therefore $G'_{i,j} = \langle w_i, w_j \rangle$. Once we solve the problem for any matrix congruent to G, we can transform the answer back using A.

Part 1: $p > 2$. Over a field of odd characteristic, every quadratic form can be diagonalized by congruence. If a nonzero diagonal entry exists, use it as a pivot and eliminate the rest of its row and column. If all diagonal entries are zero but some off-diagonal entry $G_{1,2} \neq 0$ exists, replace (v_1, v_2) by $(v_1 + v_2, v_1 - v_2)$; in odd characteristic this creates a nonzero diagonal entry. Repeating this diagonalizes the form.

Let the diagonalized matrix be

$$G' = \operatorname{diag}(g_1, \ldots, g_k, 0, \ldots, 0), \qquad g_i \neq 0.$$

Then the minimum possible dimension m is either k or $k + 1$. More precisely, $m = k$ if and only if the number of quadratic non-residues among $g_1, \ldots, g_k$ is even.

The lower bound $m \geq k$ is clear. If the number of non-residues is odd and $m = k$, let $B = [v_1, \ldots, v_n]$. Then $G' = B^T B$, so for the leading $k \times k$ part,

$$\det(G'_{[1,k]}) = \prod_{i=1}^{k} g_i = (\det B_{[1,k]})^2.$$

The right side is a quadratic residue, but the left side is a product of an odd number of non-residues, contradiction.

The construction is as follows.

- If g_i is a quadratic residue, set the i-th vector to have only one nonzero coordinate, equal to $\sqrt{g_i}$.
- Pair up the remaining non-residues. For a pair g_1, g_2, find (a, b) such that $a^2 + b^2 = g_1$. Since g_2/g_1 is a residue, let $s = \sqrt{g_2/g_1}$ and take

$$v_1 = (a, b, 0, \ldots, 0), \qquad v_2 = (bs, -as, 0, \ldots, 0).$$

  Then $\langle v_1, v_1 \rangle = g_1$, $\langle v_2, v_2 \rangle = g_2$, and $\langle v_1, v_2 \rangle = 0$.
- If one non-residue remains, realize it alone as $a^2 + b^2 = g_i$, which increases the dimension to $k + 1$.

To find a pair (a, b), it is enough to solve it for one fixed non-residue; all other cases reduce to square roots. Since many solutions exist, randomly enumerate a and check whether the corresponding b exists.

The time complexity is

$$\mathcal{O}\left(n^3 + p^{0.25} \log p \log\log p + n \log^2 p\right).$$

Part 2: $p = 2$. In characteristic 2, quadratic forms cannot always be diagonalized, so the previous method no longer works. Define

$$J = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

We claim that every matrix over $\mathbb{F}_2$ is congruent to a block diagonal matrix whose diagonal blocks are [0], [1], and J.

To obtain this block diagonal form, first handle the case with a diagonal 1 similarly to the odd-characteristic case. After that, the remaining diagonal entries are all zero. If $G_{1,2} = 1$, then the entries $G_{1,2} = G_{2,1} = 1$ can be used to eliminate all other entries in the first two rows and columns, leaving a J block. Induction completes the decomposition.

Suppose the resulting block diagonal matrix G' has a blocks [0], b blocks [1], and c blocks J. Then

$$a + b + 2c = n,$$

$$b + 2c = r = \operatorname{rank}(G') = \operatorname{rank}(G),$$

and

$$b \neq 0 \quad \iff \quad \text{some diagonal entry of } G \text{ is } 1.$$

These facts are enough to determine the answer. The minimum m is either r or $r + 1$, and $m = r$ holds if and only if $b \neq 0$.

The lower bound $m \geq r$ is clear. If $b = 0$ and $m = r$, then the vectors form a basis of the space. However, all diagonal entries are zero, so every vector has an even number of ones; their span cannot contain a vector with an odd number of ones, contradiction.

For the construction, first build $2c$ vectors of dimension $2c + 1$ for the J blocks: set $v_{i,j} = 1$ iff $j \leq i + 1$, and then change every $v_{2i,2i}$ to 0. If $b = 0$, this already finishes the construction. Otherwise, set the $(2c+1)$-st vector to have ones in the first $2c+1$ coordinates, and for each remaining [1] block set $v_{2c+i,j} = 1$ iff $j = 2c + i$. The total dimension is $2c + b$.

The time complexity is $\mathcal{O}(n^3)$, and it may be possible to improve it to $\mathcal{O}(n^3/w)$ with bitset optimization.

Problem J. Guess The Sequence 2

For a fixed way of choosing intervals, decide whether the original permutation can be recovered uniquely or whether more than one permutation is possible.

Process values x in increasing order. Consider all intervals with $m_i \leq x$. If at least two numbers among $1, 2, \ldots, x$ are not covered by these intervals, then the permutation is not unique: swapping those two numbers in the original permutation still satisfies all constraints.

Also consider the numbers not covered by intervals with $m_i < x$. After excluding the invalid case above, there are at most two such numbers. Exactly one of them must lie in the intersection of all intervals with $m_i = x$; otherwise, if two of them are both in that intersection, they can again be swapped without violating any constraint.

These two conditions are also sufficient for uniqueness. Thus define a DP state

$$f(x, p) = \text{number of valid choices after processing } x, \text{ with } p \text{ uncovered},$$

where $p = 0$ means all numbers are covered.

Let x be the current value. Let a be a number such that every value between a and x in the original permutation is at most x, and let b be a number such that between b and x there is at least one value greater than x. The transitions are:

1. $f(x-1, 0) \to f(x, 0)$: x is covered.
2. $f(x-1, 0) \to f(x, x)$: x is not covered.
3. $f(x-1, a) \to f(x, a)$: x is covered and a remains uncovered.
4. $f(x-1, a) \to f(x, 0)$: both x and a are covered, and they must be distinguishable; equivalently, the intersection of all relevant intervals must not contain a.
5. $f(x-1, b) \to f(x, b)$: x is covered.

The direct DP is $\mathcal{O}(n^2)$. Because the data are random, the total number of relevant a values over all x is $\mathcal{O}(n \ln n)$, and for a fixed x all type-5 transitions have the same coefficient. A global multiplication tag handles them efficiently, giving expected time $\mathcal{O}(n \ln n)$.

Problem K. Game: Battle of Menjis

The main conclusion is that the result is always exactly the same as if the game had only one round, no matter how many rounds are played.

Alice can, in rounds $2, 3, \ldots, k$, choose the position that Bob chose in the previous round. This restores the value decreased by Bob and makes the game equivalent to one round, so the k-round answer is at least the one-round answer. Conversely, Bob can, in rounds $1, 2, \ldots, k-1$, choose the position that Alice chose in the same round. This restores the value increased by Alice and again reduces the game to the one-round case, so the k-round answer is at most the one-round answer. Hence the two answers are equal.

Now enumerate Alice's choice and then Bob's choice. Bob's effect on the final xor depends only on the number of trailing zeroes in the binary representation of the value he chooses. Therefore, for each possible number of trailing zeroes, record whether such a value exists, and brute-force Bob's effective choices.

The time complexity is $\mathcal{O}(n \log a_i)$.

Problem L. Slay the Spire

First assume that every edge can be used. For each edge, we may pay its cost to change its starting vertex. In the end, we need to form a complete Eulerian walk. For each vertex, we must pay for at least

$$\max(0,\ \text{outdeg}(v) - \text{indeg}(v) - [v \text{ is the start}] - \#\{\text{potions ending at } v\})$$

edges that currently start from v. Choose the cheapest such edges. One can prove that after making these choices, there exists a way to change starting vertices so that every vertex satisfies

$$\text{indeg}(v) + [v \text{ is the start}] + \#\{\text{potions ending at } v\} = \text{outdeg}(v).$$

The only remaining issue is whether the final graph lies in one connected component that can be traversed continuously. An isolated component appears only when there is a strongly connected component of the initial graph with indegree zero and all of its edges are kept. In this case we must additionally pay

for at least one edge inside that strongly connected component. At the same time, we may pay for one fewer edge leaving that component. Enumerate the best such replacement for every zero-indegree strongly connected component.

The time complexity is $\mathcal{O}(m \log n)$.

Problem M. Puzzle: Summon

The linked source currently says that this analysis will be added later. No additional solution text for this problem is present in the source post.